

**Szegedi Tudományegyetem
Informatikai Intézet**

SZAKDOLGOZAT

Szabó Patrik Péter

2026

**Szegedi Tudományegyetem
Informatikai Intézet**

**PecaPont: Horgász hely foglaló és információ szerző
webalkalmazás**

Szakdolgozat

Készítette:

Szabó Patrik Péter

programtervező
informatikus szakos
hallgató

Témavezető:

Dr. Bilicki Vilmos

egyetemi adjunktus

Szeged
2026

Feladatkiírás

A horgászok számára jelenleg nehézkes egy helyen megbízható információt találni tavakról, versenyekről, hírekről és horgász helyekről. Az információk gyakran szétszórtnak, különböző weboldalakon vagy közösségi média felületeken érhetők el, ami időigényessé és kényelmetlenné teszi a tervezést.

A PecaPont célja, hogy egy központi, könnyen használható digitális platformot biztosítson, ahol a felhasználók gyorsan hozzáférhetnek minden releváns horgászati információhoz.

A feladat egy olyan webalkalmazás elkészítése Angular alapú keretrendszer használatával, ahol a felhasználók könnyedén elérnek minden releváns információt. Hozzáférnek vízterületek listájához, azoknak a részletes leírásával, tájékozódhatnak aktualitásokról, tudomást szerezhetnek versenyekről és megoszthatják sikereiket a közösséggel.

Tartalmi összefoglaló

Téma megnevezése:

PecaPont: Horgász hely foglaló és információ szerző webalkalmazás

A megadott feladat megfogalmazása:

Egy olyan webalkalmazás készítése, ahol a felhasználók könnyedén eligazodnak és megszerzik a kellő információt a horgászatuk megtervezéséhez.

Megoldási mód:

A feladat abszolválásához az Angular keretrendszert és Firebase felhő alapú szolgáltatásait használok.

Alkalmazott eszközök, módszerek:

A fejlesztés Visual Studio Code-al zajlik. A munka során Angular keretrendszert használok Typescript, HTML, SCSS nyelvekkel és Firebase felhő alapú szolgáltatásaival.

Elért eredmények:

A webalkalmazás Firebase hosting szolgáltatással elérhető az interneten, a forráskód és dokumentáció megtalálható a publikus GitHub repositóban.

A webalkalmazás reszponzív megvalósításának köszönhetően nemcsak asztali számítógépen vagy laptopon, hanem mobil eszközökön is használható. Ennek köszönhetően széleskörűen elérhetőek a szükséges információk a horgászok számára.

Kulcsszavak:

horgászat, tavak, horgászok, Angular, Firebase

Bevezetés

Az informatika és az internetes technológiák fejlődésének köszönhetően az információhoz való hozzáférés jelentősen átalakult az elmúlt évtizedekben. Ez a folyamat a horgászat világát sem kerülte el: egyre több horgászvíz üzemeltetője jelenik meg különböző online platformokon, elsősorban közösségi média felületeken. Ezek az oldalak azonban gyakran eltérő struktúrában, különböző részletességgel és nem egységes formában tartalmazzák az információkat, ami megnehezíti azok hatékony feldolgozását és összehasonlítását.

A horgászok jelentős része emiatt több forrásból próbál tájékozódni: közösségi média csoportokban, fórumokon, illetve más felhasználók beszámolóira támaszkodva gyűjtenek információt egy-egy horgászvízről. Ezek az információk azonban sok esetben szubjektívek, hiányosak vagy elavultak lehetnek, így nem minden esetben nyújtanak megbízható alapot a döntéshozatalhoz. A jelenlegi helyzet tehát egy olyan problémát vet fel, amelyben az információk szétszórtsága és változó minősége megnehezíti a horgászok számára a gyors és pontos tájékozódást.

A dolgozat célja egy olyan webalkalmazás, a PecaPont megtervezése és megvalósítása, amely egységes keretbe foglalja a horgászvizekkel kapcsolatos információkat, és egy központi platformot biztosít a felhasználók számára. Az alkalmazás célja, hogy lehetővé tegye a releváns adatok – például szabályok, módszerek, fogási tapasztalatok és egyéb fontos információk – egyszerű és gyors elérését, ezáltal támogatva a horgászok döntéshozatalát.

Fontos szempont volt a webalkalmazás egyszerű és gyors használhatósága, hogy a felhasználók könnyedén eligazodhassanak az elérhető tartalmak között. Ennek érdekében a rendszer aloldalakkal épül fel, amelyek támogatják az információk kategorizálását, így a horgászok gyorsan megtalálják a számukra releváns adatokat.

A rendszer nem csupán statikus információk megjelenítésére szolgál, hanem közösségi funkciókat is integrál. A felhasználók lehetőséget kapnak arra, hogy megosszák saját élményeiket, beszámolóikat és fogásaikat, valamint visszajelzéseket adjanak mások bejegyzéseire. Ez hozzájárul egy aktív, egymást segítő közösség kialakulásához, ahol a tapasztalatcsere révén értékes, naprakész információk keletkeznek.

A webalkalmazás tervezése során kiemelt szempont volt a felhasználóbarát működés és az átlátható felépítés. Ennek érdekében a rendszer több oldalból épül fel, kategóriára lebontva, így a felhasználók gyorsan megtalálhatják a számukra releváns tartalmakat. Emellett fontos

célkitűzés volt a reszponzív kialakítás, amely biztosítja, hogy az alkalmazás különböző eszközökön – például mobiltelefonon, tableten vagy asztali számítógépen – egyaránt hatékonyan használható legyen.

Összességében a PecaPont webalkalmazás egy olyan komplex megoldást kínál, amely egyesíti az információgyűjtést és a közösségi interakciókat, ezáltal hozzájárulva a horgászok számára elérhető információk minőségének javításához és könnyebb hozzáférhetőségéhez.

1. Piackutatás

A szakdolgozat készítésének kezdeti szakaszában piackutatást végeztem annak érdekében, hogy felmérjem a célcsoport igényeit, valamint a már létező megoldásokat és potenciális versenytársakat. A kutatás célja az volt, hogy meghatározható legyen, milyen funkciók és szolgáltatások szükségesek egy olyan webalkalmazás kialakításához, amely valódi értéket nyújt a horgászok számára.

A vizsgálat során több olyan weboldalt és webalkalmazást elemeztem, amelyek működési elve vagy célja részben vagy egészben hasonlít a PecaPont koncepciójához. Az elemzés során figyelembe vettem az egyes rendszerek funkcionalitását, felhasználói felületét, valamint az általuk kínált szolgáltatások körét. Külön figyelmet fordítottam arra, hogy ezek az alkalmazások milyen módon kezelik a horgászvizekkel kapcsolatos információkat, illetve milyen közösségi funkciókat biztosítanak.

A piackutatás másik fontos eleme egy anonim kérdőíves felmérés volt, amelynek célja a horgászok szokásainak, információszerzési módjainak és elvárásainak feltérképezése volt. A kérdőív segítségével adatokat gyűjtöttem többek között arról, hogy a felhasználók milyen platformokon keresnek információt, mennyire tartják megbízhatónak az ott talált tartalmakat, valamint milyen funkciókat tartanak hasznosnak egy ilyen jellegű alkalmazásban.

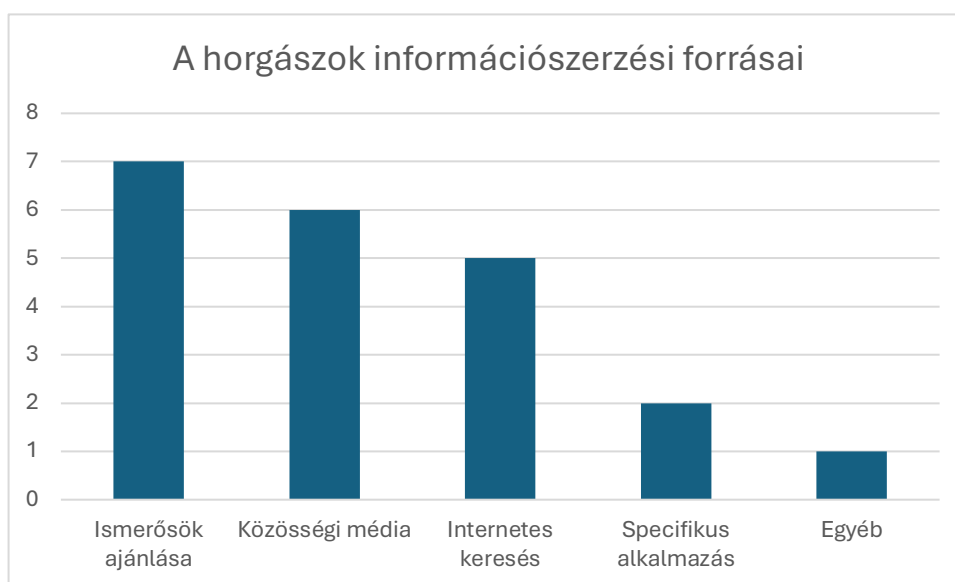
A kutatás eredményei hozzájárultak a PecaPont webalkalmazás követelményeinek meghatározásához, és alapul szolgáltak a rendszer tervezése során meghozott döntésekhez. Az így nyert információk segítségével lehetőség nyílt egy olyan alkalmazás kialakítására, amely jobban illeszkedik a felhasználói igényekhez, és hatékonyabban kezeli a horgászattal kapcsolatos információkat.

1.1. Kérdőíves felmérés eredményei

A piackutatás részeként egy anonim kérdőíves felmérést végeztem, amelynek célja a horgászok szokásainak, információszerzési módszereinek, valamint egy lehetséges webalkalmazással kapcsolatos elvárásainak feltérképezése volt. A kérdőívet különböző online felületeken tettem közzé, így a válaszadók eltérő korosztályból és tapasztalati háttérrel rendelkeztek.

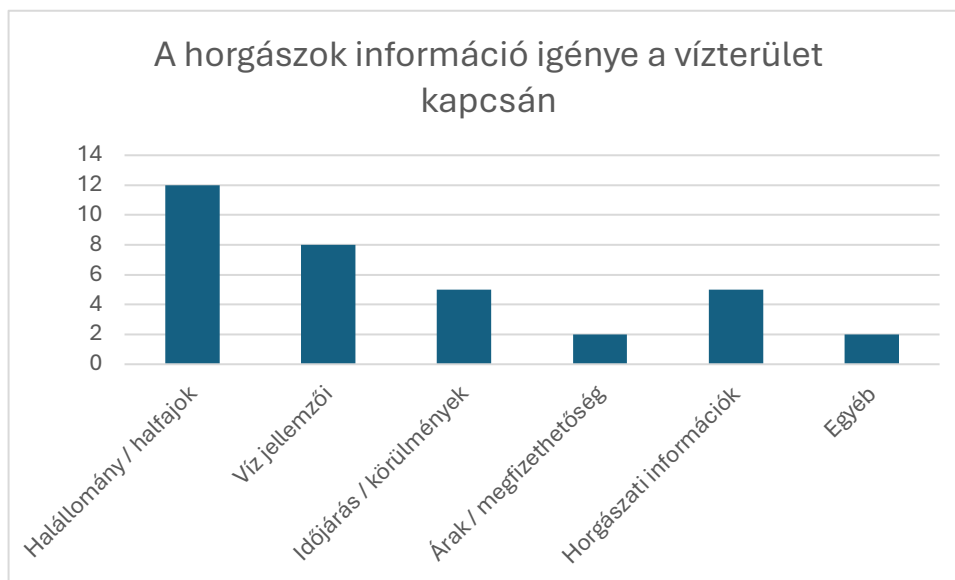
A kitöltők többsége férfi volt, és több korcsoport is képviseltette magát, ami lehetővé tette különböző felhasználói igények vizsgálatát. A válaszadók jelentős része több éves, esetenként évtizedes horgászati tapasztalattal rendelkezik, ami arra utal, hogy a célcsoport nem kizárólag kezdő horgászokból áll. Ennek megfelelően egy ilyen alkalmazásnak a tapasztaltabb felhasználók igényeit is ki kell szolgálnia.

Az információszerzési szokások vizsgálata során megállapítható volt, hogy a horgászok jelenleg több, egymástól független forrásból tájékozódnak. A leggyakoribb információforrások közé tartoznak az ismerősök ajánlásai, a közösségi média felületek, valamint az internetes keresések. Ez a gyakorlat megerősíti azt a problémát, hogy az információk szétszórtnak, nem egységes formában érhetők el.



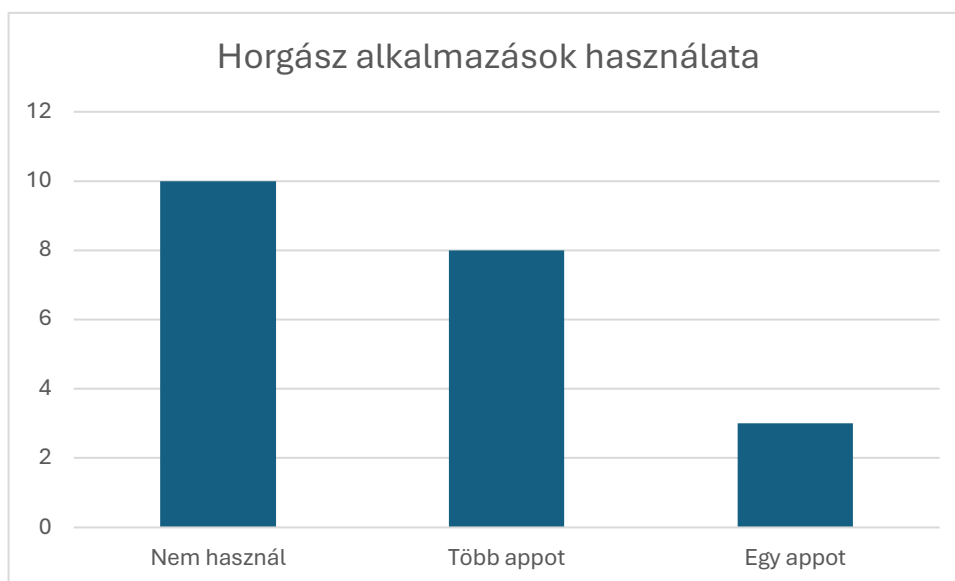
1. ábra a kérdőíves felmérés eredménye információszerzés kérdéskörben.

A válaszok alapján a horgászok számára kiemelten fontos információk közé tartozik a halállomány, a víz jellemzői, az árak, valamint az aktuális körülmények, például az időjárás vagy a víz állapota. Ezek az adatok alapvetően befolyásolják a horgász hely kiválasztását, így egy ilyen jellegű alkalmazásban központi szerepet kell, hogy kapjanak.



2. ábra a kérdőíves felmérés eredménye információigény témakörben

A jelenleg használt digitális megoldások vizsgálata során kiderült, hogy sok felhasználó nem használ kifejezetten horgászatra specializált alkalmazást, vagy több különálló platformot vesz igénybe (például időjárás-alkalmazásokat és közösségi oldalakat). Ez tovább erősíti egy egységes, több funkciót integráló rendszer iránti igényt.



3. ábra a kérdőíves felmérés eredménye alkalmazás használat témakörben

A válaszadók többsége pozitívan fogadta egy olyan webalkalmazás ötletét, amely egy helyen biztosítja a horgászvizekkel kapcsolatos információkat, valamint lehetőséget ad a tapasztalatok és fogások megosztására. Többen kiemelték, hogy hasznosnak tartanák a részletes tóinformációkat, a közösségi visszajelzéseket, valamint az esetleges helyfoglalási lehetőségeket is.

A felmérés során az is megfigyelhető volt, hogy a felhasználók kerülnek a túlzott reklámokat, a félrevezető információkat és a nem átlátható, fizetős funkciókat. Ez fontos szempontként jelent meg a rendszer tervezése során.

Összességében a kérdőív eredményei alátámasztották a PecaPont webalkalmazás létjogosultságát. A válaszok alapján egyértelmű igény mutatkozik egy olyan egységes platform iránt, amely megbízható, strukturált és könnyen hozzáférhető információkat biztosít a horgászok számára, miközben közösségi funkciókat is kínál.

1.2. Potenciális versenytársak

A piackutatás során nemcsak a felhasználói igények feltérképezése történt meg, hanem a meglévő, hasonló célú rendszerek vizsgálata is. Bár kifejezetten a PecaPont koncepciójával teljes mértékben megegyező webalkalmazás nem található, több olyan platform létezik, amelyek részben hasonló funkciókat látnak el.

A horgászok által leggyakrabban használt felületek közé tartoznak a közösségi média platformok, például a Facebook különböző csoportjai. Ezekben a csoportokban a felhasználók megoszthatják tapasztalataikat, fogásaikat, valamint információt kérhetnek másoktól. Előnyük a gyors információcsere és a nagy felhasználói bázis, azonban az információk gyakran rendezetlenek, nehezen visszakereshetők, és megbízhatóságuk változó.

Léteznek továbbá kifejezetten horgászattal foglalkozó webalkalmazások, amelyek lehetőséget biztosítanak a fogások rögzítésére, statisztikák vezetésére, valamint közösségi funkciók használatára. Ezek az alkalmazások azonban sok esetben nem tartalmazznak részletes, helyspecifikus információkat a magyarországi horgászvizekről, illetve egyes funkcióik fizetősek.

Emellett különböző információs weboldalak is rendelkezésre állnak, amelyek egy-egy horgászvízről szolgáltatnak adatokat, például szabályzatokat vagy árakat. Ezek az oldalak

azonban általában nem egységes rendszerben működnek, és ritkán kínálnak közösségi funkciókat vagy felhasználói visszajelzéseket.

Horgasz helyek.hu

A Horgasz helyek.hu egy országos szintű adatbázis, amelynek célja, hogy a magyarországi horgászvizeket egy helyen tegye elérhetővé és összehasonlíthatóvá. Az oldalon megtalálhatók a legfontosabb információk, például a halfajok, szabályok, szolgáltatások és árak, valamint felhasználói értékelések is.

Az oldal egyik legnagyobb előnye, hogy strukturált formában, jól áttekinthetően biztosít információkat a különböző vízterületekről, beleértve a halfajokat, szabályokat, szolgáltatásokat és árakat. Emellett keresési és szűrési lehetőségeket is kínál, ami megkönnyíti a felhasználók számára a megfelelő horgász hely kiválasztását.

Ugyanakkor az oldal kevésbé fókuszál a közösségi interakciókra, és nem biztosít valós idejű adatokat, például aktuális fogási információkat, így inkább statikus információforrásként működik.

Vampire Fishing

A Vampire Fishing Trips platform egy modernebb, technológiai megközelítést alkalmazó rendszer, amely elsősorban mobilalkalmazásként érhető el, és különböző digitális megoldásokkal támogatja a horgászokat.

Előnye, hogy közösségi és eseményalapú funkciókat is kínál, valamint a felhasználók aktivitására épít, ami növeli az interaktivitást.

Ellenben a rendszer kevésbé egységes felépítésű, és nem kifejezetten egy átfogó horgász hely-adatbázisra épül, így az információk nem minden esetben strukturáltak vagy könnyen összehasonlíthatók.

FishMap

A FishMap alkalmazás egy innovatív megoldás, amely térképes megjelenítéssel és valós idejű adatokkal segíti a horgászokat.

Az alkalmazás egyik legnagyobb erőssége, hogy GPS-alapú működésének köszönhetően lehetővé teszi a fogások rögzítését és azok vizualizálását, például hőtérképek segítségével. Ez jelentősen megkönnyíti az aktív horgász helyek azonosítását.

A rendszer főként a fogási adatokra koncentrál, és kevésbé nyújt részletes, strukturált információkat a horgászvizek egyéb jellemzőiről, például szabályokról vagy árakról, továbbá elsősorban mobilalkalmazásként érhető el.

1.3. Összegzés

Összességében megállapítható, hogy a vizsgált rendszerek különböző területeken erősek, azonban egyik sem kínál teljes körű, integrált megoldást. Egyes platformok strukturált adatokat biztosítanak, míg mások a közösségi vagy valós idejű információkra helyezik a hangsúlyt. A PecaPont célja ezen megközelítések egyesítése, egy olyan egységes rendszer létrehozásával, amely egyszerre biztosít strukturált információkat, közösségi funkciókat és naprakész adatokat a felhasználók számára.

Funkció	Horgász helyek,.hu	FishMap	Vampire	PecaPont
Helyadatok	igen	részleges	részleges	igen
Valós idejű fogások	nem	igen	igen	igen
Közösségi funkciók	részleges	igen	igen	igen
Egységes platform	részleges	nem	nem	igen

4. ábra a táblázat összefoglalja a vizsgált rendszerek főbb funkcióit, és összehasonlítja azokat a PecaPont webalkalmazással.

A táblázat alapján jól látható, hogy a jelenlegi megoldások különböző területeken nyújtanak erős funkcionalitást, azonban egyik sem kínál teljes körű megoldást. A Horgasz helyek.hu elsősorban strukturált helyadatokat biztosít, de nem rendelkezik valós idejű információkkal. A FishMap és a Vampire platformok támogatják a valós idejű fogások megjelenítését és a közösségi funkciókat, ugyanakkor nem biztosítanak egységes, átfogó információs rendszert. Ezzel szemben a PecaPont célja, hogy egyetlen platformon egyesítse a

hely adatokat, a közösségi funkciókat és a valós idejű információkat, így komplexebb megoldást nyújtva a felhasználók számára.

2. Funkcionális specifikáció

A piackutatás és a versenytársak elemzése alapján meghatározhatók azok a funkcionális és nem funkcionális követelmények, amelyeknek a PecaPont webalkalmazásnak meg kell felelnie. A kérdőíves felmérés eredményei rámutattak arra, hogy a horgászok számára kiemelten fontos a gyors, megbízható és strukturált információelérés, valamint a közösségi funkciók jelenléte. Emellett a versenytársak vizsgálata azt is megmutatta, hogy a jelenlegi megoldások nem kínálnak teljes körű, egységes platformot.

2.1. Funkcionális követelmények

A funkcionális követelmények a rendszer által biztosított konkrét szolgáltatásokat írják le.

A rendszernek biztosítani kell a horgászvizek adatainak megjelenítését, beleértve a halállományt, a víz jellemzőit, az árakat és a szabályokat. A felhasználóknak képesnek kell lenniük a horgász helyek közötti keresésre és szűrésre különböző paraméterek alapján, például név, település vagy víztípus szerint.

A rendszernek támogatnia kell a felhasználói tartalmak kezelését. Ennek keretében a regisztrált felhasználók képesek fogásokat rögzíteni, bejegyzéseket létrehozni, valamint értékeléseket és hozzászólásokat írni a horgász helyekhez.

A rendszernek biztosítani kell a közösségi interakciókat, beleértve a felhasználók közötti kommunikációt és a tartalmak megosztását. A felhasználóknak lehetőséget kell biztosítani más felhasználók tartalmainak megtekintésére.

A rendszernek képesnek kell lennie valós idejű vagy közel valós idejű információk megjelenítésére, például friss fogási adatok vagy aktuális környezeti információk esetében.

A rendszernek biztosítani kell a felhasználói fiókok kezelését, beleértve a regisztrációt, bejelentkezést, kijelentkezést, valamint a profiladatok módosítását.

A funkcionális követelmények közvetlenül a piackutatás eredményeiből és a versenytársak elemzéséből kerültek levezetésre.

2.2. Nem funkcionális követelmények

A nem funkcionális követelmények a rendszer működésének minőségi jellemzőit határozzák meg.

A webalkalmazásnak felhasználóbarát és könnyen kezelhető felülettel kell rendelkeznie, amely lehetővé teszi a felhasználók számára a gyors és hatékony navigációt. A rendszernek rezponzív kialakításúnak kell lennie, hogy különböző eszközökön, például mobiltelefonon, tableten és asztali számítógépen is megfelelően működjön.

A rendszernek biztosítani kell a gyors betöltési időt és a stabil működést, mivel a felhasználók elvárják a zökkenőmentes használatot. Emellett fontos a megbízhatóság és az adatok pontossága, különösen a felhasználók által generált tartalmak esetében.

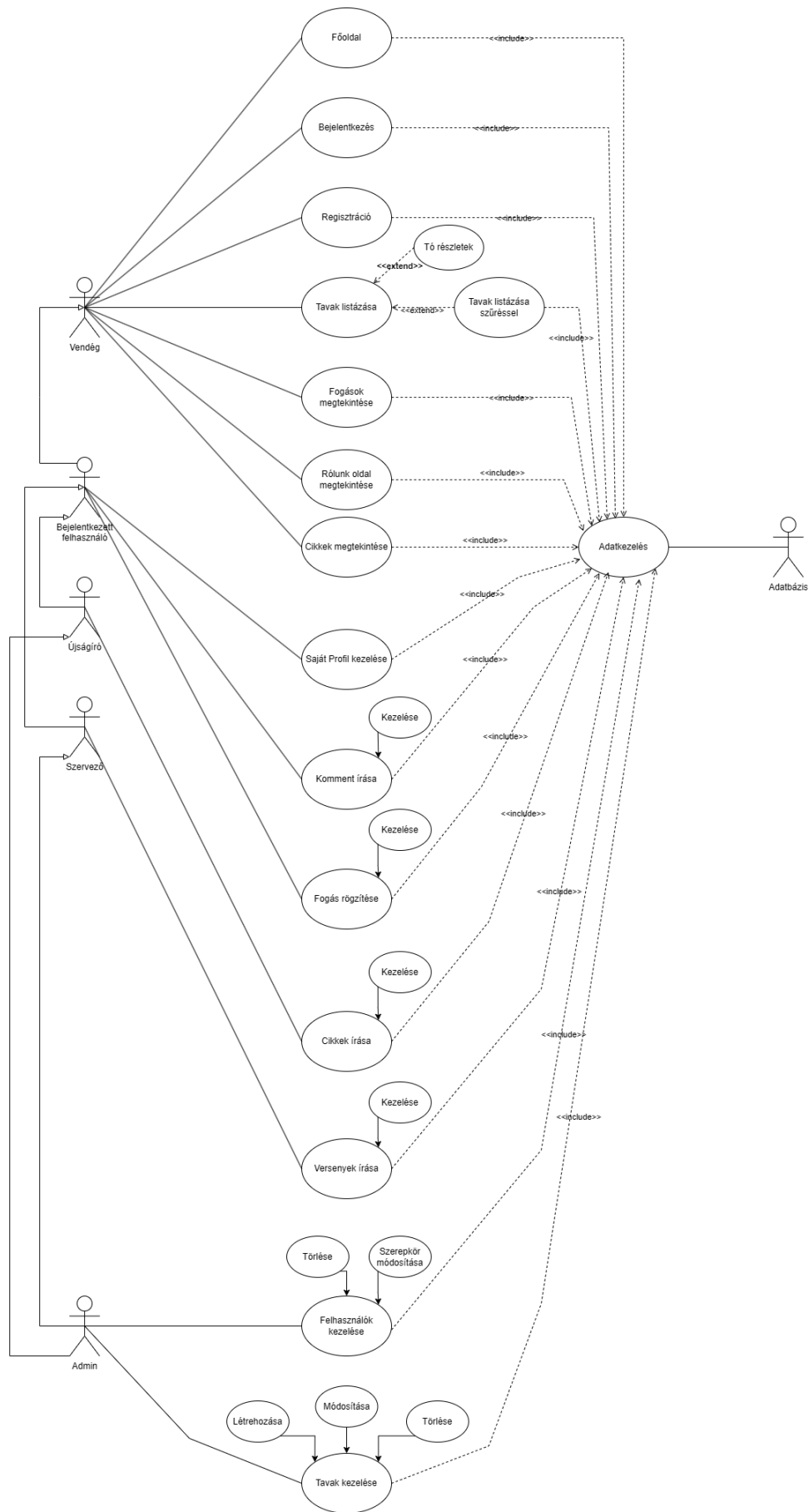
Biztonsági szempontból a rendszernek védenie kell a felhasználói adatokat, és biztosítani kell a jogosultságkezelést, hogy a különböző funkciók csak megfelelő hozzáféréssel legyenek elérhetők.

2.3. Használati eset diagram

A rendszer működésének szemléltetésére használati eset diagram készült, amely bemutatja a rendszer és a felhasználók közötti kapcsolatokat, valamint a főbb funkcionálisakat.

A diagramon öt fő szereplő jelenik meg: a vendég felhasználó, a bejelentkezett felhasználó, az újságíró, a szervező és az adminisztrátor. A vendég felhasználók számára elsősorban a böngészési és keresési funkciók érhetők el, míg a bejelentkezett felhasználók további műveleteket végezhetnek, például fogásokat rögzíthetnek vagy hozzászólásokat

írhatnak. Az újságíró cikket hozhat létre míg a szervező versenykiírásokat tehet közzé. Az adminisztrátor jogosult a rendszer adatainak kezelésére és a felhasználók adminisztrálására.



5. ábra A PecaPont webalkalmazás használati eset diagramja

2.4. Felhasználói szerepkörök

2.4.1. Vendég felhasználó

A vendég felhasználók regisztráció és bejelentkezés nélkül is használhatják a webalkalmazást, azonban funkcionalitásuk korlátozott. Lehetőségük van a horgászhelyek böngészésére, keresésére és szűrésére, valamint a horgászhelyek adatlapjainak megtekintésére. Hozzáférnek a feltöltött fogásokhoz, valamint az alap oldalakhoz. Emellett lehetőségük van regisztrációra és bejelentkezésre.

2.4.2. Bejelentkezett felhasználó

A bejelentkezett felhasználók a rendszer teljes funkcionalitásához hozzáférnek. A horgászhelyek böngészése és megtekintése mellett lehetőségük van fogások rögzítésére, hozzászólások írására, értékelések küldésére, valamint saját profiljuk kezelésére.

2.4.3. Újságíró

Az újságíró szerepkörrel rendelkező felhasználók speciális jogosultságokkal rendelkeznek a rendszerben. A bejelentkezett felhasználókhöz képest további lehetőségük van cikkek létrehozására és kezelésére. Ezek a cikkek információkat tartalmazhatnak például horgászati tippekről, beszámolókról vagy hírekről, amelyek a többi felhasználó számára is elérhetők.

2.4.4. Szervező

A szervező szerepkörrel rendelkező felhasználók jogosultak horgászversenyek létrehozására és kezelésére. A versenyek kiírása során megadhatják a szükséges adatokat, például a helyszínt, időpontot és a részvételi feltételeket. Ez a funkció lehetővé teszi a közösségi események szervezését és lebonyolítását a platformon belül.

2.4.5. Adminisztrátor

Az adminisztrátor jogosult a rendszer adatainak kezelésére, statisztikák elérésére. Feladatai közé tartozik a felhasználók kezelése, a horgászhelyek adatainak karbantartása, valamint a nem megfelelő tartalmak moderálása.

3. Tervezett megjelenés

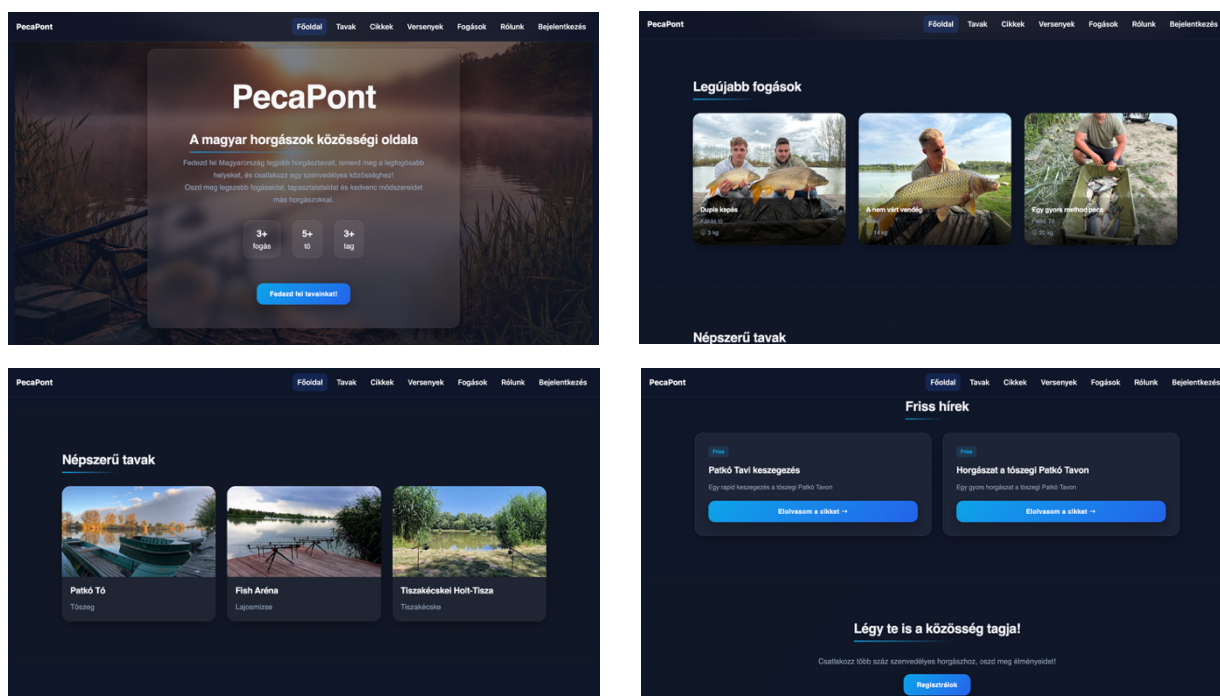
A webalkalmazás felhasználói felületének megtervezése során kiemelt szempont volt az átláthatóság, az egyszerű használhatóság, valamint a reszponzív megjelenés biztosítása. A képernyőtervek kialakítása során a modern webalkalmazások felépítését vettem alapul, különös figyelmet fordítva a felhasználói élményre.

Az alkalmazás egy egységes megjelenési móddal rendelkezik, amely a sötét témára épül, ugyanakkor különböző fényviszonyok mellett is jól használható. A felület kialakításánál a konzisztens színhasználat és az egységes komponensstruktúra volt meghatározó.

A reszponzív működés biztosítása érdekében a felület különböző képernyőméretekre lett optimalizálva, így az alkalmazás mobil eszközökön és tableteken is megfelelően használható. A fejlesztés során a különböző nézetek kialakítása során figyelembe vettem a kisebb kijelzők korlátait, például az elemek átrendeződését és a navigáció egyszerűsítését.

3.1. Főoldal

A felhasználók az alkalmazás megnyitásakor a főoldalra jutnak, amely áttekintést nyújt az alkalmazás tartalmáról. A főoldal célja, hogy gyors navigációs lehetőséget biztosítson a különböző funkciók eléréséhez, valamint kiemelje a fontosabb tartalmakat.



6. ábra A főoldal kinézete

3.2. Tavak oldal

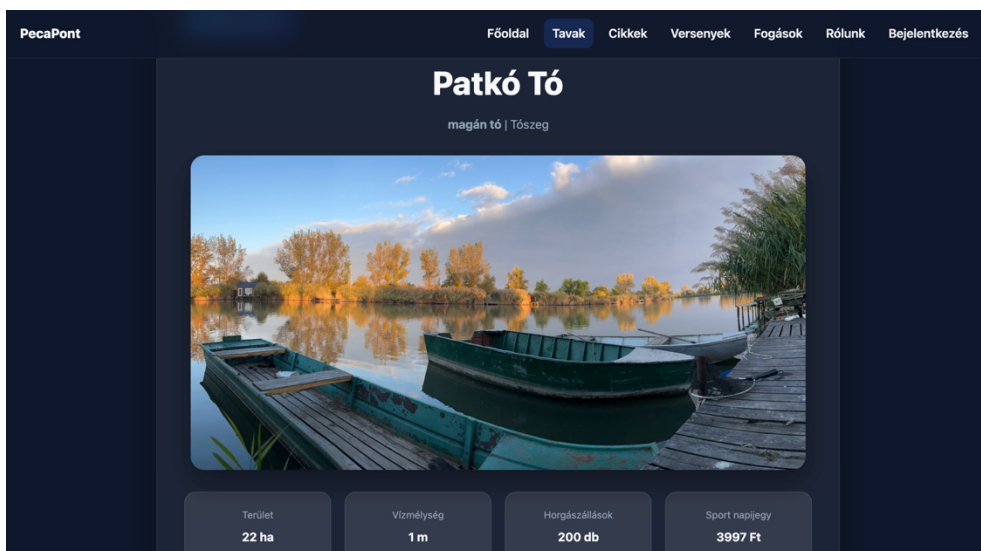
A tavak oldalon a felhasználók a rendszerben szereplő horgász helyek között böngészhetnek. Az elemek kártyás megjelenítésben jelennek meg, ahol az egyes tavak nevei és településük látható. A felhasználó egy adott elem kiválasztásával eljut a részletes nézetre.



7. ábra A tavak oldal kinézete

3.3. Tó részletes oldal

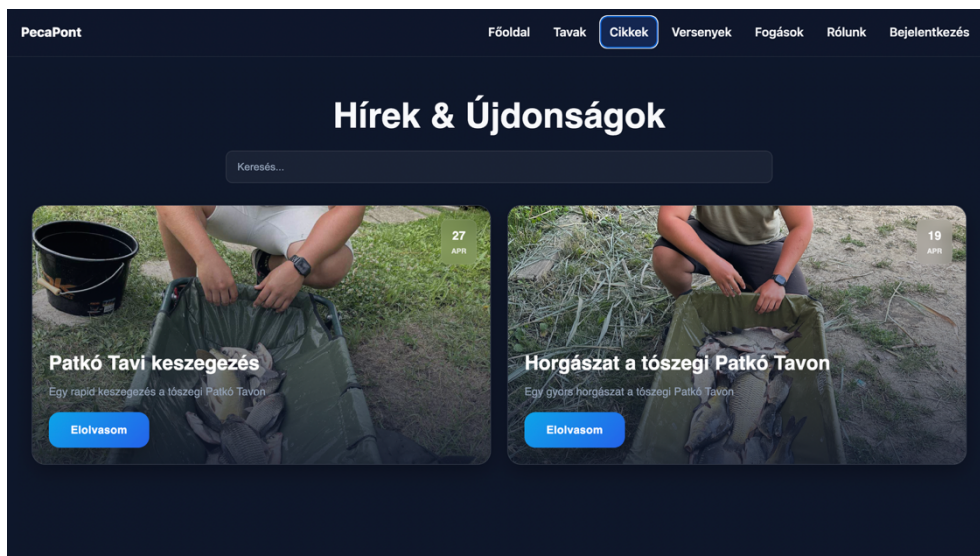
A tó részletes oldalán a kiválasztott vízterülethez tartozó részletes információk jelennek meg. Ez a nézet részletesebb leírást és további adatokat tartalmaz, amelyek segítik a felhasználót a tájékozódásban.



8. ábra Egy tó részletes kinézete

3.4. Hírek oldal

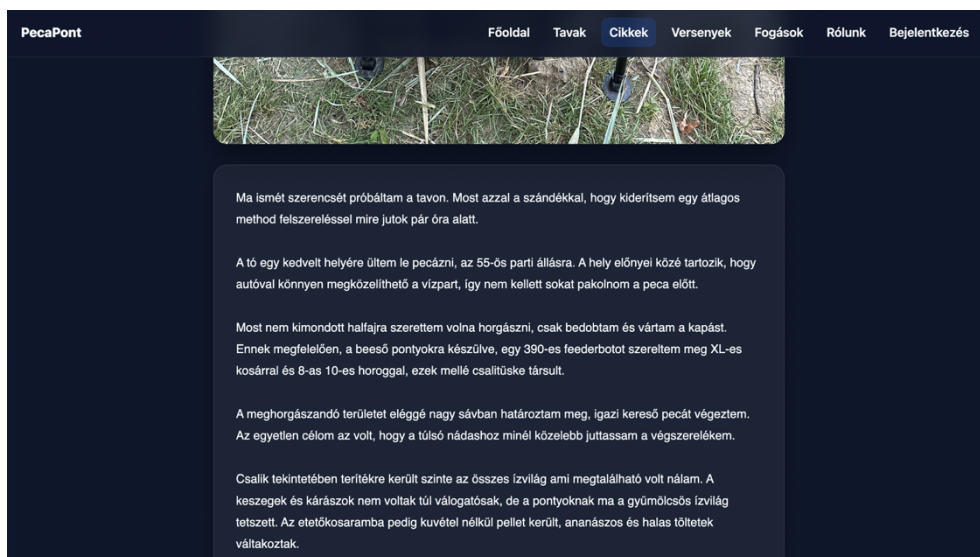
A hírek oldalon a horgászathoz kapcsolódó információk és bejegyzések jelennek meg. A hírek kártyás formában kerülnek megjelenítésre, és a felhasználó egy adott hír kiválasztásával részletesebb információkat tekinthet meg.



9. ábra A hírek oldal kinézete

3.5. Hír részletes oldal

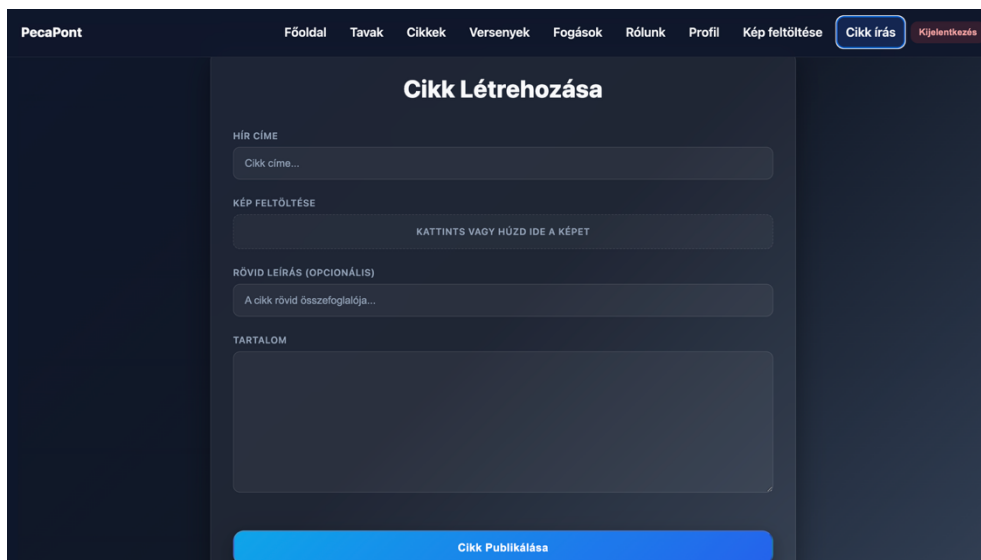
A hír részletes oldalán a kiválasztott cikkhez tartozó részletes információk jelennek meg. Ez a nézet részletesebb leírást és további adatokat tartalmaz, amelyek segítik a felhasználót a tájékozódásban.



10. ábra Egy hír részletes nézetének kinézete

3.6. Hírszerkesztő oldal

A hírszerkesztő felület kizárólag jogosultsággal rendelkező felhasználók számára érhető el. Ezen az oldalon lehetőség van új hírek létrehozására, valamint meglévő tartalmak szerkesztésére.



11. ábra Hírszerkesztő oldal kinézete

3.7. Versenyek oldal

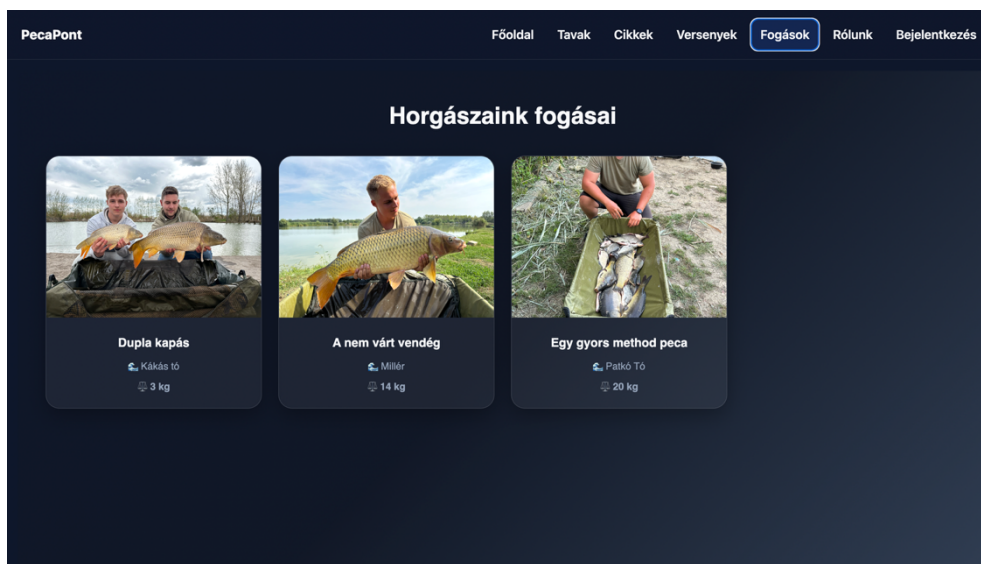
A versenyek oldalon a meghirdetett események kerülnek megjelenítésre. A felhasználók böngészhetnek a különböző versenyek között, és részletes információkat tekinthetnek meg az egyes eseményekről.

3.8. Verseny részletes oldal

A részletes verseny oldalon a felhasználók a kiválasztott esemény teljes leírásával, pontos részleteivel ismerkedhetnek meg.

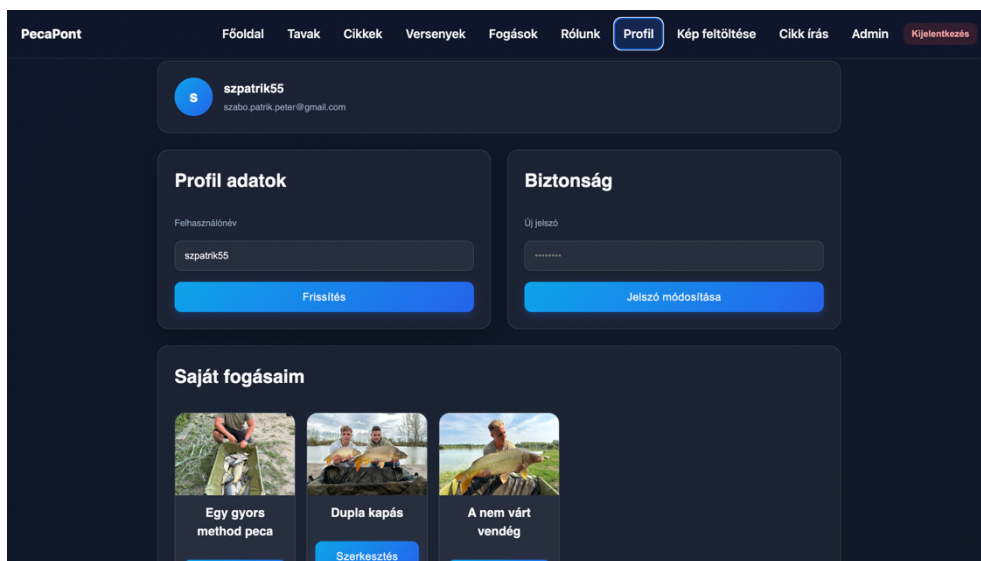
3.9. Galéria oldal

A galéria oldalon a felhasználók által feltöltött fogások jelennek meg. A tartalmak minden felhasználó számára elérhetőek, míg új képet csak bejelentkezett felhasználók tölthetnek fel.



3.10. Profil oldal

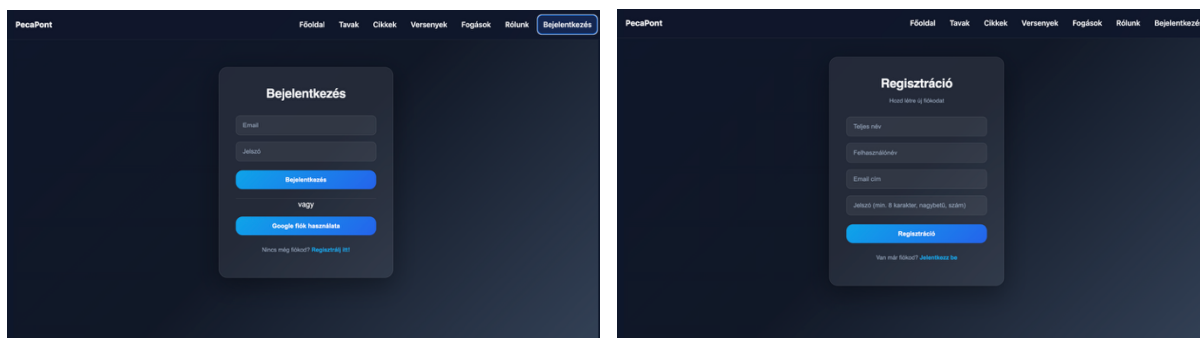
A profil oldalon a felhasználók megtekinthetik saját adataikat, valamint módosíthatják azokat. Ez a felület kizárólag hitelesített felhasználók számára érhető el.



3.11. Bejelentkezés és Regisztráció oldal

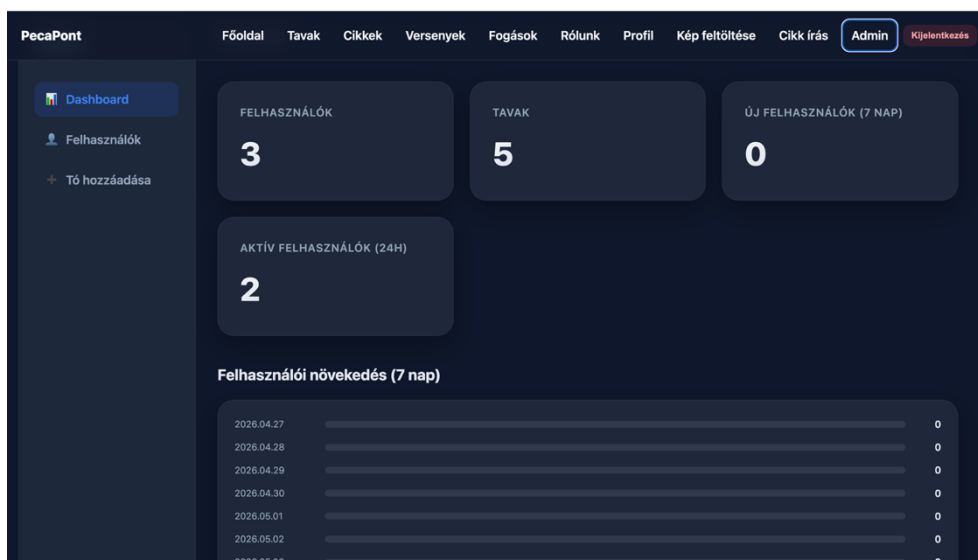
PecaPont: Horgászhely foglaló és információ szerző webalkalmazás

A felhasználó számára lehetőség van regisztrációra és bejelentkezésre. A regisztráció során a felhasználók megadják a szükséges adatokat, amelyeket a rendszer a Firebase Authentication segítségével kezel. A bejelentkezés sikeressége az adatok helyességétől függ.



3.12. Admin felület

Az adminisztrációs felület kizárólag adminisztrátori jogosultsággal rendelkező felhasználók számára érhető el. Az admin felület lehetőséget biztosít a rendszer tartalmának kezelésére, például felhasználók vagy egyéb adatok módosítására, valamint rendelkezik egy statisztikai oldallal is.



4. Felhasznált technológiák

4.1. Angular

Az Angular egy nyílt forráskódú, TypeScript alapú frontend keretrendszer, amelyet elsősorban komplex webalkalmazások fejlesztésére használnak. Az Angular komponens alapú felépítése lehetővé teszi az alkalmazás moduláris felépítését, ami megkönnyíti a karbantarthatóságot és az újrafelhasználhatóságot.

A PecaPont webalkalmazás fejlesztése során az Angular biztosította a kliensoldali logikát, a felhasználói felület kezelését, valamint az egyes funkciók strukturált megvalósítását.

4.2. Firebase

A Firebase egy felhőalapú platform, amely számos szolgáltatást kínál webes alkalmazások fejlesztéséhez. A projekt során több Firebase szolgáltatást is felhasználtam:

- **Firestore (adatbázis):** a horgász helyek, felhasználók és fogások adatainak tárolására.
- **Cloud Storage:** képek és egyéb fájlok tárolására.
- **Hosting:** az alkalmazás publikálására.

A Firebase használatával lehetőség nyílt egy skálázható és valós idejű adatkezelést támogató rendszer kialakítására, amely gyors fejlesztést és egyszerű üzemeltetést biztosít.

4.3. GitHub

A GitHub egy verziókezelő rendszerre épülő platform, amely lehetővé teszi a forráskód nyomon követését és kezelését. A fejlesztés során a GitHub segítségével történt a projekt verziókezelése, amely biztosította a módosítások követhetőségét és a fejlesztési folyamat strukturálását.

4.4. ChatGPT

A fejlesztés során a ChatGPT mesterséges intelligencia alapú eszközt használtam különböző problémák megoldására, kódpéldák generálására, valamint dokumentációs szövegek megfogalmazására. Az eszköz jelentősen hozzájárult a fejlesztési folyamat gyorsításához és a hatékonyabb munkavégzéshez.

4.5. Gemini AI

A Gemini AI egy mesterséges intelligencia alapú szolgáltatás, amely különböző szöveges műveletek elvégzésére alkalmas. A projekt során a Gemini API használatával lehetőség nyílt

szövegek generálására és feldolgozására, amely hozzájárulhat a felhasználói élmény javításához és az alkalmazás intelligens funkcióinak bővítéséhez.

4.6. AngularFire

Az AngularFire az Angular és a Firebase közötti integrációt biztosító hivatalos könyvtár. Segítségével egyszerűen megvalósítható a Firebase szolgáltatások, például az adatbázis-kezelés, hitelesítés és fájlkezelés használata Angular környezetben.

A projekt során az AngularFire könyvtárat használtam a Firestore adatbázis elérésére, az adatok lekérdezésére és módosítására, valamint a Firebase szolgáltatásokkal való kommunikáció megvalósítására.

4.7 Draw.io

A Draw.io (diagrams.net) egy online diagramkészítő eszköz, amely lehetővé teszi különböző típusú diagramok, például UML diagramok, folyamatábrák és rendszertervek elkészítését.

A szakdolgozat során a Draw.io segítségével készítettem el a használati eset diagramot, valamint a rendszer működését szemléltető további ábrákat.

5. Architektúra

A webalkalmazás architektúrája két fő részből áll: a frontend rétegből és a felhőalapú backend szolgáltatásokból. A rendszer nem hagyományos szerver–kliens architektúrát használ, hanem a Firebase platformra épülő Backend as a Service (BaaS) megoldást.

A frontend és a backend közötti kommunikáció a Firebase szolgáltatásain keresztül történik. A frontend közvetlenül éri el az adatbázist és egyéb backend funkciókat az AngularFire könyvtár segítségével, így nincs szükség külön backend szerver implementálására.

5.1. Frontend

A frontend résszel lép interakcióba a felhasználó a webalkalmazás használata során. Ez a réteg felelős a felhasználói felület megjelenítéséért és a felhasználói interakciók kezeléséért.

A frontend az Angular keretrendszer segítségével került megvalósításra, amely komponens alapú felépítést biztosít. Az alkalmazás különböző oldalakból és komponensekből épül fel, például navigációs menüből, listaoldalakból és adatlapokból.

A webalkalmazás struktúráját HTML sablonok határozzák meg, míg a megjelenésért és stílusért a SCSS felel. Az alkalmazás működéséhez szükséges logika TypeScript nyelven lett implementálva, amely kezeli a felhasználói eseményeket és az adatok lekérését.

5.2. Backend

A backend funkciókat a Firebase platform biztosítja, amely felhőalapú szolgáltatásokat kínál a webalkalmazás számára.

A Firestore adatbázis felelős az alkalmazás adatainak tárolásáért, beleértve a horgászhelyeket, felhasználókat és fogásokat. A Cloud Storage szolgáltatás segítségével a felhasználók által feltöltött képek kerülnek tárolásra.

A Firebase Authentication biztosítja a felhasználók hitelesítését, lehetővé téve a regisztrációt és bejelentkezést. Az alkalmazás hosztolása szintén a Firebase Hosting szolgáltatáson keresztül történik.

A frontend közvetlenül kommunikál ezekkel a szolgáltatásokkal az AngularFire könyvtár segítségével, amely leegyszerűsíti az adatkezelést és a backend műveletek végrehajtását.

6. Belső felépítés

Az alkalmazás Angular keretrendszerrel készült, amely a fejlesztés során standalone komponens alapú felépítést használ. Az Angular újabb verzióiban (17+) a standalone komponensek kiemelt szerepet kapnak, így az alkalmazás modulok használata nélkül is felépíthető.

A standalone komponensek lehetővé teszik, hogy a szükséges komponenseket, direktívákat és pipe-okat közvetlenül importáljuk, ami egyszerűbbé és átláthatóbbá teszi az alkalmazás struktúráját.

6.1. Komponensek (Component)

A webalkalmazás több önálló komponensből épül fel, amelyek egymásba ágyazva alkotják a teljes felhasználói felületet. A komponensek tartalmazhatnak további komponenseket, direktívákat, pipe-okat és szolgáltatásokat (service).

A komponensek két fő csoportra oszthatók:

Page komponensek

A Page komponensek olyan komponensek, amelyek különálló oldalként jelennek meg az alkalmazásban, és URL-en keresztül elérhetők. Ezek a komponensek felelősek az adott oldal teljes felépítéséért és működéséért.

UI / újrafelhasználható komponensek

Ezek a komponensek a Page komponenseken belül kerülnek felhasználásra, és kisebb, újrafelhasználható egységeket alkotnak.

A PecaPont webalkalmazásban az alábbi Page komponenseket hoztam létre:

- *app:*
A webalkalmazás gyökér komponense, a RouterOutlet segítségével dinamikusan tölti be az aktuálisan megjelenítendő oldalt.
- *home:*
A webalkalmazás főoldala, amely áttekintést nyújt a rendszer főbb funkcióiról és tartalmairól. Ez tölt be bejelentkezés és regisztráció, valamint kijelentkezés után is.

- *regisztracio:*
A webalkalmazás regisztrációs oldala. A felhasználók számára biztosít lehetőséget új fiók létrehozására, email és jelszó páros megadásával, valamint Google fiók használatával.
- *bejelentkezés:*
A webalkalmazás bejelentkezés oldala. Itt tudnak a felhasználók a felületre bejelentkezni.
- *tavak:*
A webalkalmazás Tavak oldala. A horgász helyek listáját jeleníti meg, ahol a felhasználók böngészhetnek és szűrhetnek a különböző tavak között.
- *to-reszletek:*
A webalkalmazás egy adott tavát részletesen megjelenítő oldal. Egy kiválasztott horgász hely részletes adatait jeleníti meg, vízterület méretét, szabályokat, ajánlott módszereket.
- *to-hozzaad:*
A webalkalmazás tó hozzáadása oldala. Az admin szerepkörrel rendelkező felhasználók számára nyújt lehetőséget új vízterületek felvételéhez.
- *hirek:*
A webalkalmazás Cikkek oldala. A felhasználók itt tudnak böngészni és keresni a cikkek között.
- *hir-reszletek:*
A webalkalmazás egy adott cikkjét részletező oldal. Egy adott cikk részletes megjelenítéséért felel, megjelennek a részletek és a teljes tartalom.
- *hir-szerkeszto:*
A webalkalmazás cikkek írásáért felelős oldala. Az admin, valamint újságíró szerepkörrel rendelkező felhasználók érik el és itt tudnak cikkeket közzétenni a felhasználók számára.
- *versenyek:*
A webalkalmazás Versenyek oldala. Az aktuális horgászversenyek listáját jeleníti meg. A felhasználók kereshetnek és rendezhetik a tartalmat.
- *verseny-reszletek:*
A webalkalmazás egy adott versenyt megjelenítő oldal. A verseny részletes leírását, időpontját és fontos információt jeleníti meg.

- *verseny-szerkeszto:*
A webalkalmazás versenyek publikálására szolgáló oldala. Az admin, valamint szervező szerepkörrel rendelkező felhasználók érhetik el és itt tudnak új versenyeseményeket közzétenni a felhasználók számára.
- *fogasok:*
A webalkalmazás Fogások oldala. A felhasználók itt láthatják egymás fogásait, képeit, amelyekhez csatolhatnak rövid- és hosszú leírást, a fogás típusát és súlyát vagy hosszát.
- *rolunk:*
A webalkalmazás Rólunk oldala. Egy bemutatkozó oldal, ahol a felhasználók jobban megismerhetik az oldal koncepcióját.
- *profil:*
A webalkalmazás Profil oldala. Itt tudják a felhasználók módosítani az adataikat, valamint törölni a fiókjukat.
- *admin-dashboard:*
A webalkalmazás admin panelje. Az admin szerepkörrel rendelkező felhasználók részére készült áttekintő felület, amely statisztikai adatokat jelenít meg a rendszer működéséről, például a felhasználók számát, a horgász helyek számát, valamint az aktív és új felhasználók számát. A dashboard segíti az adminisztrátort a rendszer állapotának nyomon követésében.
- *admin-layout:*
A webalkalmazás admin panelének váza. Az admin felület alapstruktúráját biztosító komponens, amely az adminisztrációs oldalak közös elrendezéséért felel.
- *admin-users:*
A webalkalmazás admin panelének felhasználók kezelésére szolgáló része. Az admin szerepkörrel rendelkező felhasználók számára lehetőséget biztosít a felhasználók kezelésére. A komponens megjeleníti a felhasználók listáját, és lehetőséget ad a szerepkörök módosítására, ezáltal támogatva a jogosultságkezelést a rendszerben.

A PecaPont webalkalmazásban az alábbi újrafelhasználható komponenseket hoztam létre:

- *navbar:*
A webalkalmazás fő navigációs eleme. Lehetővé teszi az oldalak közötti navigációt. A komponens minden oldalon megjelenik, biztosítva az egységes felhasználói élményt.

- *footer:*

A webalkalmazás alsó részén megjelenő komponens. Egységes információkat tartalmaz, a komponens minden oldalon megjelenik, biztosítva a konzisztens felhasználói felületet.

6.2 Szolgáltatások (Servicek)

A szolgáltatások (service-ek) az alkalmazás üzleti logikáját és az adatkezelést megvalósító osztályok. Feladatuk a komponensek és a backend szolgáltatások közötti kommunikáció biztosítása.

Az Angular keretrendszer dependency injection mechanizmusának segítségével a szolgáltatások könnyen újrafelhasználhatók és több komponensben is elérhetők.

A PecaPont webalkalmazásban a szolgáltatások elsősorban a Firebase alapú adatbázissal való kommunikációért, valamint az alkalmazás logikai működéséért felelősek.

A PecaPont webalkalmazásban az alábbi szolgáltatásokat hoztam létre:

- *admin:*

Az admin service az adminisztrációs funkciók megvalósításáért felelős szolgáltatás. A szolgáltatás a Firebase Firestore adatbázissal kommunikál, és lehetővé teszi a felhasználók és horgász helyek adatainak lekérését és kezelését. A szolgáltatás segítségével lekérhetők a felhasználók adatai, módosíthatók a felhasználói szerepkörök, valamint törölhetők a felhasználók. Emellett különböző statisztikai adatok lekérésére is lehetőséget biztosít, például a felhasználók és horgász helyek számának meghatározására. További funkcióként képes meghatározni az új felhasználók számát az elmúlt időszakban, az aktív felhasználók számát, valamint napi bontásban megjeleníteni a regisztrációk alakulását. Ezek az adatok az adminisztrációs felületen jelennek meg, segítve a rendszer állapotának áttekintését.

- *auth:*

Az auth service a felhasználók hitelesítéséért és jogosultságkezeléséért felelős szolgáltatás. A szolgáltatás a Firebase Authentication és a Firestore adatbázis együttes használatával valósítja meg a felhasználók kezelését. A szolgáltatás figyeli az aktuális autentikációs állapotot, és ennek megfelelően lekéri a felhasználóhoz tartozó adatokat a

Firestore adatbázisból. Ez lehetővé teszi, hogy az alkalmazás a bejelentkezett felhasználóhoz tartozó kiegészítő információkat, például szerepkört és profiladatokat is kezelje. A szolgáltatás támogatja az e-mail és jelszó alapú regisztrációt és bejelentkezést, valamint a Google fiókkal történő hitelesítést is. A regisztráció során a felhasználó adatai bekerülnek az adatbázisba, beleértve a létrehozás időpontját és a felhasználó szerepkörét. A bejelentkezés során a szolgáltatás frissíti a felhasználó utolsó belépésének időpontját, amely később statisztikai célokra is felhasználható. A szolgáltatás továbbá kezeli a felhasználói jogosultságokat is. Külön Observable értékeken keresztül biztosítja annak ellenőrzését, hogy az adott felhasználó rendelkezik-e adminisztrátori vagy szerkesztői, szervezői jogosultságokkal. Ez lehetővé teszi a különböző funkciókhoz való hozzáférés szabályozását az alkalmazásban.

- *gallery:*

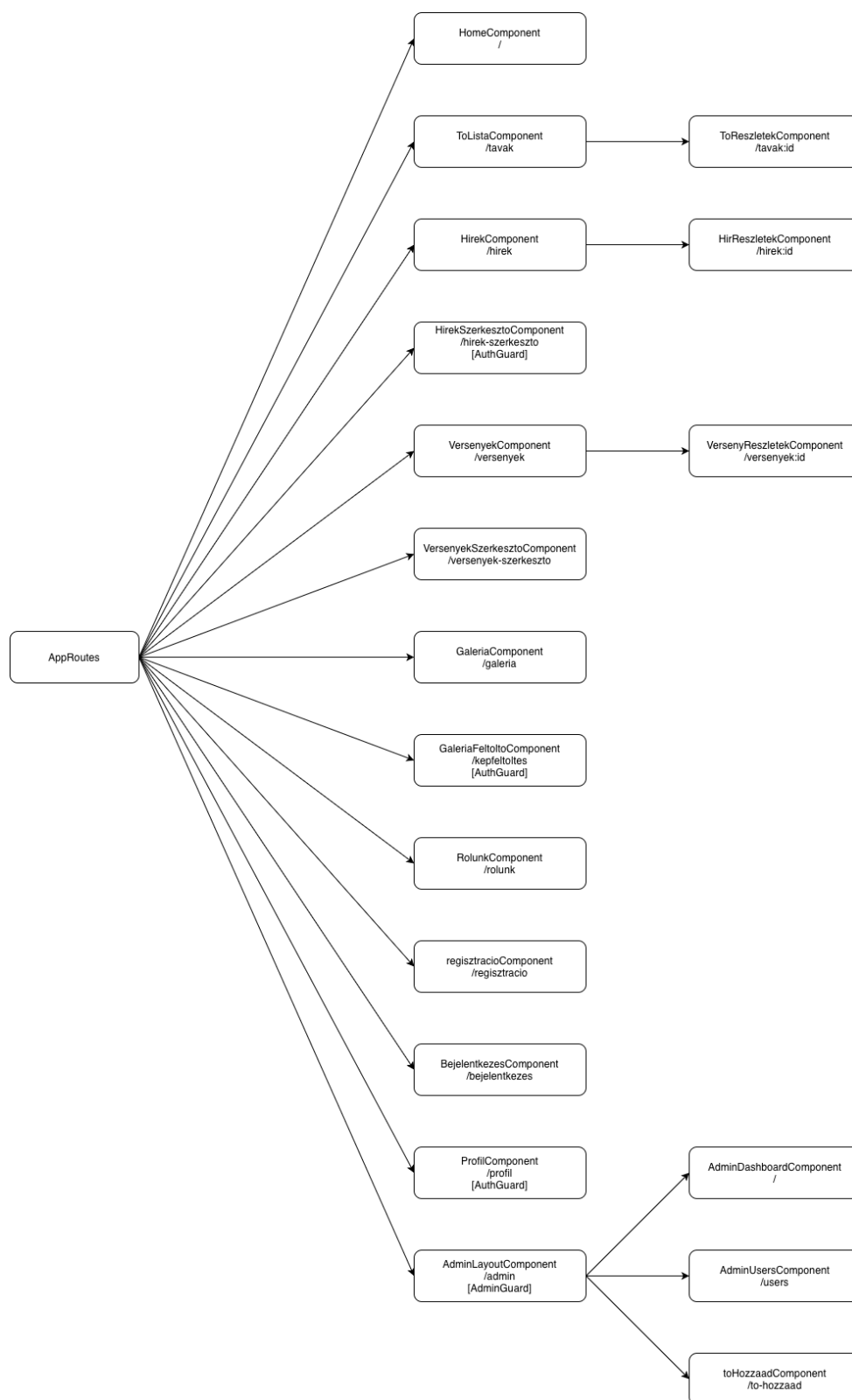
A gallery service a felhasználók által feltöltött képek és galéria tartalmak kezeléséért felelős szolgáltatás. A szolgáltatás a Firebase Cloud Storage és a Firestore adatbázis együttes használatával valósítja meg a képfeltöltést és a kapcsolódó adatok tárolását. A szolgáltatás lehetővé teszi képfájlok feltöltését a felhőbe, majd a feltöltött fájlok elérési útjának (URL) lekérését. Ez az URL később felhasználható a képek megjelenítésére a webalkalmazásban. Emellett a szolgáltatás biztosítja a galéria bejegyzések létrehozását is, amelyek tartalmazhatják a feltöltött képekhez kapcsolódó adatokat. Ezek az adatok a Firestore adatbázisban kerülnek tárolásra. A szolgáltatás aszinkron műveleteket használ, és az Angular zónakezelésének segítségével biztosítja, hogy a felhasználói felület megfelelően frissüljön a feltöltési folyamat során.

- *news:*

A news service a hírek és cikkek kezeléséért felelős szolgáltatás. A szolgáltatás a Firebase Firestore adatbázist és a Cloud Storage szolgáltatást használja a tartalmak tárolására és kezelésére. A szolgáltatás lehetővé teszi képek feltöltését a hírekhez, amelyeket a rendszer a felhőben tárol, majd a hozzájuk tartozó elérési út (URL) visszaadásra kerül. Ez az URL később felhasználható a hírek megjelenítése során. A szolgáltatás támogatja új hírek létrehozását, amelyek a Firestore adatbázisban kerülnek tárolásra. A hírek létrehozásakor a rendszer automatikusan rögzíti a létrehozás időpontját, amely lehetővé teszi az adatok időrendi rendezését. A szolgáltatás továbbá biztosítja az összes hír lekérdezését, valamint

az egyedi hírek elérését azonosító alapján. A hírek lekérdezése időrendi sorrendben történik, így a legfrissebb tartalmak jelennek meg elsőként a felhasználói felületen.

6.7. Routing



A webalkalmazás felületei közötti navigáció megvalósítása az Angular keretrendszer Router moduljának segítségével történik. A Router feladata az URL-ek feldolgozása és azok megfelelő komponensekhez történő leképezése, amely lehetővé teszi a dinamikus, oldalújrátöltés nélküli navigációt. Ennek eredményeként az alkalmazás egyoldalas alkalmazásként (Single Page Application, SPA) működik.

A routing konfiguráció során az egyes funkcionális egységek külön útvonalakhoz kerültek hozzárendelésre. A kezdőoldal a gyökér útvonalon (/) érhető el, míg az egyéb tartalmak – például a tavak listája (/tavak) és az egyes tavak részletes nézete (/tavak/:id) – paraméterezett útvonalakon keresztül jelennek meg. Hasonló módon a hírek megjelenítése is támogatja az egyedi azonosítóval ellátott részletező nézeteket (/hirek/:id).

A felhasználói interakciókhoz kapcsolódó funkciók, mint például a regisztráció (/regisztracio), bejelentkezés (/bejelentkezés) és profilkezelés (/profil) szintén külön útvonalakon érhetők el. A galéria funkciók esetében elkülönül a tartalom megjelenítése (/galeria) és a képfeltöltés (/kepfeltoltes).

Az alkalmazás biztonságának biztosítása érdekében bizonyos útvonalak védelmét route guard-ok segítségével valósítottam meg. Az authGuard használatával kizárólag hitelesített felhasználók férhetnek hozzá olyan funkciókhoz, mint például a profil oldal, a képfeltöltés vagy a hírszerkesztő felület. Ezzel szemben az adminGuard további jogosultsági szintet biztosít, amely kizárólag adminisztrátori jogosultsággal rendelkező felhasználók számára engedélyezi az adminisztrációs felület elérését.

Az adminisztrációs modul megvalósítása lazy loading technikával történt, amelynek célja az alkalmazás teljesítményének optimalizálása. Az admin felület (/admin) és annak aloldalai – például a felhasználókezelés (/admin/users) és az új tartalom hozzáadása (/admin/to-hozzaad) – csak akkor töltődnek be, amikor a felhasználó ténylegesen navigál ezekre az oldalakra. Ez csökkenti a kezdeti betöltési időt és javítja a felhasználói élményt.

A routing konfiguráció végén egy ún. „wildcard” útvonal került definiálásra (**), amely minden nem létező útvonal esetén a kezdőoldalra irányítja vissza a felhasználót, ezáltal biztosítva az alkalmazás robusztus működését.

7. Biztonság

A webalkalmazás biztonsági megoldásai több rétegben kerültek kialakításra, amelyek központi eleme a Firebase Cloud Firestore adatbázisra definiált biztonsági szabályrendszer. A Firestore lehetőséget biztosít finomhangolt hozzáférés-kezelésre, amely segítségével meghatározható, hogy az egyes kollekciókhoz milyen műveletek hajthatók végre különböző felhasználói szerepkörök esetén.

A rendszer megkülönbözteti a hitelesített (bejelentkezett) felhasználókat a vendégektől, ezáltal eltérő hozzáférési jogosultságokat biztosít számukra. A többféle szerepkör alkalmazásának köszönhetően a hitelesített felhasználók jogosultságai között is különbség tehető, amely lehetővé teszi a részletes jogosultságkezelés megvalósítását.

C = Létrehozás R = Olvasás U = Módosítás D = Törlés	Felhasználó	Fogások	Hírek	Versenyek	Értékelések	Komment	Fogások
Vendég	CR	R	R	R	R	R	
Felhasználó	RUD	CRUD	R	R	CRUD	CRUD	
Újságíró	RUD	CRUD	CRUD	R	CRUD	CRUD	
Szervező	RUD	CRUD	R	CRUD	CRUD	CRUD	
Admin	RUD	CRUD	CRUD	CRUD	CRUD	CRUD	

12. ábra Kollektiókhoz és erőforrásokhoz való hozzáférés

A kollekciókhoz és erőforrásokhoz tartozó hozzáférési jogosultságokat a 6. ábra szemlélteti. A vendég felhasználók elsősorban olvasási jogosultsággal rendelkeznek, míg a hitelesített felhasználók a szerepkörüktől függően további műveletek végrehajtására is jogosultak, például tartalmak létrehozására és módosítására.

A Firestore biztonsági szabályok között kiemelt szerepet kap a felhasználók hitelesítettségének ellenőrzése, amely a következő feltétel segítségével történik: `request.auth != null`. A hitelesítés ellenőrzésén túl a rendszer további biztonsági mechanizmusokat is alkalmaz, amelyek a felhasználók szerepkörén és az adatok tulajdonjogán alapulnak. A Firestore szabályok között definiálásra kerültek olyan segédfüggvények, amelyek egyszerűsítik és egységesítik az ellenőrzések végrehajtását.

A szerepkör alapú jogosultságkezelés megvalósításához egy külön függvény került kialakításra, amely a felhasználó szerepkörét a users kollekcióból olvassa ki, és összehasonlítja a szükséges jogosultsággal. Ennek segítségével meghatározható, hogy egy adott művelet végrehajtásához a felhasználó rendelkezik-e megfelelő jogosultsággal.

```

1 rules_version = '2';
2
3 service cloud.firestore {
4   match /databases/{database}/documents {
5
6     function isLoggedin() {
7       return request.auth != null;
8     }
9
10    // Segédfüggvény a szerepkör ellenőrzéséhez
11    function hasRole(role) {
12      return isLoggedin() &&
13        get(/databases/{database}/documents/users/{request.auth.uid}).data.role == role;
14    }
15
16    // Tavak: Bárki olvashatja, csak admin módosíthatja
17    match /lakes/{doc} {
18      allow read: if true;
19      allow write: if hasRole('admin');
20    }
21
22    // Galéria: Tulajdonos vagy Admin törölhet/módosíthat
23    match /gallery/{doc} {
24      allow read: if true;
25      allow create: if isLoggedin() && request.resource.data.uid == request.auth.uid;
26      allow update, delete: if isLoggedin() && (request.auth.uid == resource.data.uid || hasRole('admin'));
27    }
28
29    // Hírek: Bárki olvashat, Admin/News írhat, törölhet
30    match /news/{newsId} {
31      allow read: if true;
32      allow create, update, delete: if
33        hasRole('admin') || hasRole('news');
34    }
35
36    // Versenyek: Bárki olvashat, Admin/Organiser írhat, törölhet
37    match /events/{eventsId} {
38      allow read: if true;
39      allow create, update, delete: if
40        hasRole('admin') || hasRole('organiser');
41    }
42
43    match /users/{userId} {
44      // Először azt nézzük meg, hogy a saját adata-e.
45      // Ha igen, az OR (||) második fele (hasRole) le sem fut, így nincs hiba.
46      allow read: if isLoggedin() && (request.auth.uid == userId || hasRole('admin'));
47
48      allow create: if isLoggedin() && request.auth.uid == userId;
49
50      allow update: if isLoggedin() && request.auth.uid == userId
51        && request.resource.data.role == resource.data.role;
52
53      allow delete: if hasRole('admin');
54    }
55  }
56 }

```

13. ábra Firestore biztonsági szabályok

Az egyes kollekciókhoz tartozó szabályok eltérő működést valósítanak meg. A lakes kollekció esetében az adatok olvasása minden felhasználó számára engedélyezett, azonban módosítási műveleteket kizárólag adminisztrátori jogosultsággal rendelkező felhasználók végezhetnek.

A gallery kollekció esetében a rendszer biztosítja, hogy új tartalom létrehozása csak hitelesített felhasználók számára legyen elérhető, valamint ellenőrzi, hogy a létrehozott dokumentum a megfelelő felhasználóhoz tartozik. A módosítás és törlés kizárólag a tartalom tulajdonosa vagy adminisztrátor által hajtható végre, amely megakadályozza más felhasználók adatainak illetéktelen kezelését.

A news és events kollekciók esetében a hozzáférés szerepkör-alapú módon kerül szabályozásra. Míg az adatok olvasása minden felhasználó számára engedélyezett, addig a létrehozás, módosítás és törlés csak meghatározott szerepkörökhöz kötött.

A users kollekció kezelése kiemelt biztonsági szempontokat követ. A felhasználók kizárólag a saját adataikhoz férhetnek hozzá, és azok módosítása során nem változtathatják meg saját szerepkörüket. Ez a megkötés megakadályozza a jogosultsági szintek jogosulatlan módosítását.

A backend oldali biztonsági megoldásokat kiegészítik a frontend oldalon alkalmazott védelmi mechanizmusok. Az Angular alkalmazás route guard-okat használ annak érdekében, hogy bizonyos oldalak kizárólag megfelelő jogosultsággal rendelkező felhasználók számára legyenek elérhetők. Az authGuard a bejelentkezett állapotot ellenőrzi, míg az adminGuard az adminisztrátori jogosultság meglétét vizsgálja. Emellett az alkalmazás űrlapvalidációkat is alkalmaz, amelyek biztosítják a felhasználói adatok helyességét és teljességét.

8. Adatmodell

Adatok tárolására Google Firebase Cloud Firestore szolgáltatást alkalmaztam, míg a fájlok kezelésére a Firebase Storage szolgáltatás került felhasználásra.

A Cloude Firestore NoSQL típusú adatbázis, amelyben az adatok dokumentumok formájában kerülnek eltárolásra, és ezek kollekciókba szerveződnek. A relációs adatbázisokkal ellentétben a NoSQL nem használ hagyományos táblák közötti kapcsolatokat, hanem az adatok közötti kapcsolatok logikai szinten kerülnek megvalósításra.

8.2. Tavak (Lakes)

A tavak kollekcióban a rendszerben megjelenített horgász helyek adatai kerülnek eltárolásra. A tavakat minden felhasználó megtekintheti, azonban módosításuk kizárólag adminisztrátori jogosultsággal lehetséges.

Egy vízterülethez eltárolásra kerül annak neve, leírása, elhelyezkedése, valamint egyéb releváns adatok. A dokumentumok egyedi azonosítóval rendelkeznek, amely lehetővé teszi azok egyértelmű azonosítását.

8.3. Galéria (Gallery)

A galéria kollekcióban a felhasználók által feltöltött képek és azok leírásai kerülnek tárolásra. A tartalmak minden felhasználó számára elérhetőek, azonban feltöltés kizárólag bejelentkezett felhasználók számára engedélyezett.

Egy galéria elem tartalmazza a feltöltő felhasználó azonosítóját, a kép elérési útvonalát, valamint a képen szereplő fogás adatait. A képek fizikai tárolása a Firebase Storage szolgáltatásban történik.

8.3. Hírek (News)

A hírek kollekcióban az alkalmazásban megjelenő információs tartalmak találhatóak. A hírek minden felhasználó számára elérhetőek, azonban létrehozásuk és módosításuk szerepkörhöz kötött.

Egy hírhez eltárolásra kerül a cím, a tartalom, a létrehozás dátuma, valamint a szerző azonosítója.

8.4. Versenyek (Events)

A versenyek kollekció tartalmazza a meghirdetett eseményeket. Ezek az adatok minden felhasználó számára elérhetőek, azonban létrehozásuk és módosításuk kizárólag megfelelő jogosultsággal rendelkező felhasználók számára engedélyezett.

Egy versenyhez eltárolásra kerül annak neve, időpontja, helyszíne, leírása, valamint a szervező azonosítója.

8.5. Felhasználók (Users)

A felhasználók kollekcióban a regisztrált felhasználók adatai kerülnek tárolásra. Az autentikációs adatokat a Firebase Authentication kezeli, míg a további felhasználói adatok ebben a kollekcióban kerülnek eltárolásra.

Egy felhasználóról eltárolásra kerül az egyedi azonosító (uid), név, e-mail cím, felhasználónév, valamint szerepkör.